

"I have a Python script called run_experiments.py that trains and evaluates models. It already has a block for ProcessTransformer. I need to add a similar block for my CNN model called MyktbekModel. The block should: instantiate MyktbekModel with vocab_size=len(vocab), embed_dim=64, num_filters=128, num_activities=len(vocab), dropout=0.1. Then train it using train_myktbek_model. Then run inference on encoded_test in batches of 64 using torch.no_grad(). Convert predicted indices to activity names using idx_to_act dictionary. Denormalize time predictions using std_val and mean_val. Finally call evaluate_by_prefix_length and save_results. Can you write this block?"

 LISA Pro

Рассуждение заняло 4 секунд(ы) ▾

Here's the code block for your MyktbekModel following the same structure as your ProcessTransformer block:

python

⌵ Свернуть Запустить Копировать

```
# ===== MyktbekModel (CNN) =====
print("\n" + "="*60)
print("Training MyktbekModel (CNN)...")
print("="*60)

# Instantiate model
myktbek_model = MyktbekModel(
    vocab_size=len(vocab),
    embed_dim=64,
    num_filters=128,
    num_activities=len(vocab),
    dropout=0.1
)

# Train model
myktbek_model = train_myktbek_model(myktbek_model, encoded_train, encoded_train_time,
                                     epochs=EPOCHS, batch_size=BATCH_SIZE)

# Run inference on test set
myktbek_pred_activities = []
myktbek_pred_times = []
```

```

myktbek_model.eval()
with torch.no_grad():
    for i in range(0, len(encoded_test), 64):
        batch_encoded = encoded_test[i:i+64]
        batch_tensor = torch.LongTensor(batch_encoded).to(device)

        # Get predictions
        pred_activities, pred_times = myktbek_model(batch_tensor)

        # Convert to numpy and collect
        myktbek_pred_activities.extend(pred_activities.cpu().numpy())
        myktbek_pred_times.extend(pred_times.cpu().numpy())

# Convert predicted indices to activity names
myktbek_pred_activity_names = [idx_to_act[idx] for idx in myktbek_pred_activities]

# Denormalize time predictions
myktbek_pred_times_denorm = [(t * std_val) + mean_val for t in myktbek_pred_times]

# Evaluate by prefix length
myktbek_results = evaluate_by_prefix_length(
    myktbek_pred_activity_names,
    myktbek_pred_times_denorm,
    test_data
)

# Save results
save_results(myktbek_results, "myktbek_cnn_results.json")

print("MyktbekModel training and evaluation complete!")

```

This block follows the same pattern as your ProcessTransformer code:

- **Model instantiation** with your specified parameters
- **Training** using `train_myktbek_model`
- **Batched inference** (batch size 64) with `torch.no_grad()`
- **Activity name conversion** using `idx_to_act`
- **Time denormalization** using `std_val` and `mean_val`
- **Evaluation and saving** via `evaluate_by_prefix_length` and `save_results`

Make sure you have all the required variables (`encoded_train` , `encoded_train_time` , `encoded_test` , `test_data` , `EPOCHS` , `BATCH_SIZE` , `device` , `idx_to_act` , `std_val` , `mean_val`)

defined in scope before this block runs.

The inference code has some issues. `encoded_test` is already a PyTorch tensor, so I should not wrap it in `torch.LongTensor()` again — just use `.to(device)` directly. Also `pred_activities` returns logits not indices, so I need `torch.max` to get the predicted index. And `pred_indices` should be converted with `.tolist()` and then `int()` before looking up in `idx_to_act`. Can you fix the inference block only?

 LISA Pro

Рассуждение заняло 3 секунд(ы) ▾

Here's the corrected inference block:

python

⌵ Свернуть Запустить Копировать

```
# Run inference on test set
myktbek_pred_activities = []
myktbek_pred_times = []

myktbek_model.eval()
with torch.no_grad():
    for i in range(0, len(encoded_test), 64):
        batch_encoded = encoded_test[i:i+64].to(device)

        # Get predictions
        pred_activities, pred_times = myktbek_model(batch_encoded)

        # Get predicted indices from logits using torch.max
        _, pred_indices = torch.max(pred_activities, dim=1)

        # Convert to list and collect
        myktbek_pred_activities.extend(pred_indices.tolist())
        myktbek_pred_times.extend(pred_times.cpu().numpy())

# Convert predicted indices to activity names
myktbek_pred_activity_names = [idx_to_act[int(idx)] for idx in myktbek_pred_activities]

# Denormalize time predictions
myktbek_pred_times_denorm = [(t * std_val) + mean_val for t in myktbek_pred_times]
```

Key fixes:

1. **No double wrapping** – `encoded_test` is already a tensor, so just call `.to(device)` directly
2. **Logits to indices** – Added `torch.max(pred_activities, dim=1)` to get the predicted class indices from the logits
3. **Proper conversion** – Used `.tolist()` on the tensor, then `int(idx)` when looking up in `idx_to_act`

